

CYBERSHIELD

Phishing URL Detection System

Cybersecurity Student Project

Student: _____

Guide/Trainer: _____

Institute: _____

Submission Date: _____

Abstract

CyberShield is a defensive URL-analysis application that examines a URL without visiting it. It extracts structural features such as URL length, HTTPS usage, IP-address hosts, subdomain count, suspicious keywords, special characters, URL shorteners and punycode. A transparent rule-based scoring engine converts these indicators into a 0–100 risk score and classifies the URL as Legitimate, Suspicious or Phishing. The application also explains the indicators behind the decision. A synthetic labelled dataset and a separate hand-written test set are included for academic validation. The implementation uses Python and Streamlit and is organized so that students can explain each major part during a viva. The synthetic evaluation is only a demonstration of the implementation and is not evidence of real-world phishing-detection performance.

1. Introduction

Phishing can use deceptive links and domains to trick users. A URL can contain structural signals that justify additional scrutiny. This project focuses on pre-visit URL analysis rather than opening or interacting with a suspicious website.

2. Problem Statement

Users may have difficulty distinguishing legitimate URLs from deceptive ones. The project provides a simple tool that accepts a URL, extracts meaningful features, calculates a transparent risk score, and presents a classification with reasons.

3. Objectives

1) Accept a URL. 2) Extract URL-based cybersecurity features. 3) Apply transparent detection rules. 4) Return a clear classification and risk score. 5) Explain the indicators behind the decision. 6) Validate the detector with labelled sample data and test cases.

4. Scope

In scope: lexical/structural URL analysis, risk scoring, explanation, synthetic dataset evaluation, and a Streamlit interface. Out of scope: visiting URLs, malware execution, DNS investigation, browser automation, live reputation lookups, credential collection, and unauthorized testing.

5. Literature / Existing System

Existing anti-phishing solutions can use browser warnings, reputation databases, machine-learning models, DNS information and webpage analysis. Those approaches may require large datasets, external services, or infrastructure. CyberShield uses transparent local rules so students can understand and demonstrate each decision.

6. Proposed System

Workflow: input URL → normalize → parse hostname/path/query → extract features → apply weighted rules → calculate score → classify → display reasons and features. The detector does not open the URL.

7. System Requirements

Hardware: ordinary student laptop/desktop. OS: Windows/Linux/macOS. Python 3.10+. Streamlit interface. pandas, scikit-learn and optional plotting libraries. Core parsing uses Python standard-library modules.

8. Methodology / Implementation

The feature extractor checks HTTPS, IP-address hostnames, '@' symbols, URL length, subdomains, suspicious keywords, hyphens, special characters, URL shorteners and punycode. Each indicator contributes predefined risk points. The total is capped at 100. Scores below 30 are Legitimate, 30–59 Suspicious, and 60–100 Phishing.

9. Testing

The project includes 15 hand-written test URLs covering normal domains, IP hosts, '@' obfuscation, URL shorteners, account keywords, multiple subdomains and special characters. The synthetic dataset contains 120 labelled URLs.

10. Results & Analysis

The synthetic dataset was evaluated as a binary security-flag task: Phishing is positive, Legitimate is negative, and both Suspicious and Phishing detector outputs count as a positive flag. Results: Accuracy=1.000, Precision=1.000, Recall=1.000, F1=1.000. Confusion matrix: TN=60, FP=0, FN=0, TP=60. Because the data is synthetic and designed around the implemented rules, these metrics must not be presented as real-world performance.

11. Limitations

Rule-based systems can miss novel patterns and can flag benign URLs containing suspicious-looking features. The dataset is synthetic, so it lacks the diversity and adversarial behavior of real-world data. The system does not inspect page content or live domain reputation.

12. Future Scope

Use a legally obtained real-world labelled dataset, train an explainable machine-learning classifier, add batch CSV analysis, incorporate offline threat-intelligence feeds, and compare models using cross-validation.

13. Conclusion

CyberShield demonstrates a complete defensive cybersecurity workflow: URL input, feature extraction, explainable detection, testing, and reporting. It is simple enough to demonstrate and explain during a viva.

References

Python documentation; Streamlit documentation; scikit-learn documentation; uploaded Cybersecurity Student Project Submission Guidelines; synthetic dataset created for this academic project.